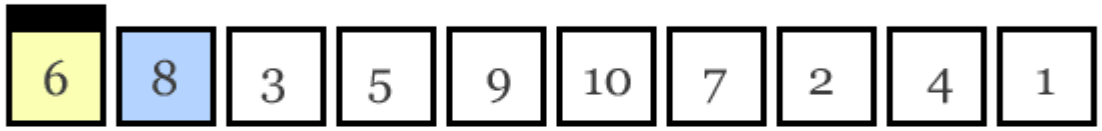
SELECTION AND MERGE SORTING : LECTURE

DSAA 211: WEEK 9



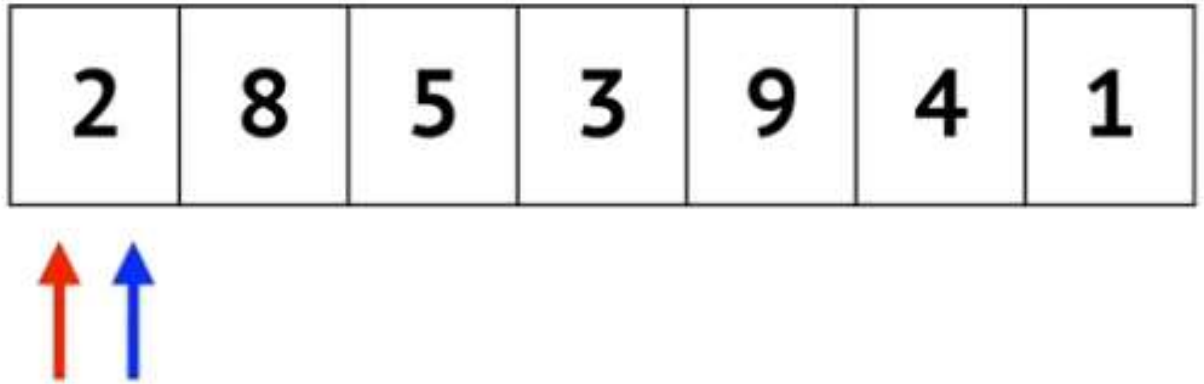
SELECTION SORTING

The selection sort algorithm sorts an array by repeatedly finding the minimum element (considering ascending order) from the unsorted part and putting it at the beginning.



Yellow is smallest number found
Blue is current item
Green is sorted list

HOW SELECTION SORT WORKS?



1. Start with the first element as the initial position.
2. Find the smallest element in the unsorted portion of the array.
3. Swap this smallest element with the first unsorted element.
4. Move the boundary of the sorted portion one element forward.
5. Repeat steps 2-4 for the remaining unsorted elements until the entire array is sorted.

EXAMPLE:

```
//29, 10, 14, 37, 13
```

Pass	Unsorted Portion	Smallest Value	Action Taken	Resulting Array
1	29, 10, 14, 37, 13	10	Swap 29 and 10	10, 29, 14, 37, 13
2	29, 14, 37, 13	13	Swap 29 and 13	10, 13, 14, 37, 29
3	14, 37, 29	14	Already smallest → no swap	10, 13, 14, 37, 29
4	37, 29	29	Swap 37 and 29	10, 13, 14, 29, 37

TIME COMPLEXITY

Feature	Description
Type	Comparison-based sorting
In-place?	Yes (does not need extra memory)
Stable?	No (relative order of equal elements may change)
Time Complexity	$O(n^2)$ — even if the list is almost sorted
Best use case	Small lists or when simplicity is more important than speed

GENERAL ALGORITHM

1. **Start** with the first element in the array.
2. **Assume** it is the smallest element.
3. **Compare** this element with all other elements in the unsorted portion of the array.
4. **Find the smallest element** in the unsorted portion.
5. **Swap** the smallest element found with the first element of the unsorted portion.
6. **Move the boundary** between sorted and unsorted portions one element to the right.
7. **Repeat** steps 2–6 until the entire array is sorted.

```
for i = 0 to n-2 do
    min_index = i
    for j = i+1 to n-1 do
        if array[j] < array[min_index] then
            min_index = j
    end for
    swap(array[i], array[min_index])
end for
```

PRACTICE

Unsorted List:

```
[25, 12, 22, 64, 11]
```

Pass	Unsorted Portion	Smallest Value Found	Action (Swap with index ...)	Array After This Pass
1	25, 12, 22, 64, 11	_____	_____	_____
2	_____	_____	_____	_____
3	_____	_____	_____	_____
4	_____	_____	_____	_____

PRACTICE

Unsorted List:

```
[25, 12, 22, 64, 11]
```

Pass	Unsorted Portion	Smallest Value Found	Action (Swap with index ...)	Array After This Pass
1	25, 12, 22, 64, 11	11	Swap 25 with 11	11, 12, 22, 64, 25
2	12, 22, 64, 25	12	No swap (already smallest)	11, 12, 22, 64, 25
3	22, 64, 25	22	No swap	11, 12, 22, 64, 25
4	64, 25	25	Swap 64 with 25	11, 12, 22, 25, 64

GAME 1

Unsorted List:

[45, 12, 88, 32, 5, 60, 19, 25]

Pass	Unsorted Portion	Smallest Value Found	Action (Swap with index …)	Array After This Pass
1	45, 12, 88, 32, 5, 60, 19, 25	5	Swap 45 with 5	5, 12, 88, 32, 45, 60, 19, 25
2				
3				
4				
5				
6				
7				

GAME 1

Unsorted List:

[45, 12, 88, 32, 5, 60, 19, 25]

Pass	Unsorted Portion	Smallest Value Found	Action (Swap with index ...)	Array After This Pass
1	45, 12, 88, 32, 5, 60, 19, 25	5	Swap 45 with 5	5, 12, 88, 32, 45, 60, 19, 25
2	12, 88, 32, 45, 60, 19, 25	12	No swap	5, 12, 88, 32, 45, 60, 19, 25
3	88, 32, 45, 60, 19, 25	19	Swap 88 with 19	5, 12, 19, 32, 45, 60, 88, 25
4	32, 45, 60, 88, 25	25	Swap 32 with 25	5, 12, 19, 25, 45, 60, 88, 32
5	45, 60, 88, 32	32	Swap 45 with 32	5, 12, 19, 25, 32, 60, 88, 45
6	60, 88, 45	45	Swap 60 with 45	5, 12, 19, 25, 32, 45, 88, 60
7	88, 60	60	Swap 88 with 60	5, 12, 19, 25, 32, 45, 60, 88

GAME 2

Unsorted List:

[42, 17, 8, 33, 56, 21, 19, 4, 28, 12]

Pass	Unsorted Portion	Smallest Value Found	Action (Swap with index ...)	Array After This Pass
1	42, 17, 8, 33, 56, 21, 19, 4, 28, 12			
2				
3				
4				
5				
6				
7				
8				
9				

GAME 2

Unsorted List:

[42, 17, 8, 33, 56, 21, 19, 4, 28, 12]

Pass	Unsorted Portion	Smallest Value Found	Action (Swap with index ...)	Array After This Pass
1	42, 17, 8, 33, 56, 21, 19, 4, 28, 12	4	Swap 42 with 4	4, 17, 8, 33, 56, 21, 19, 42, 28, 12
2	17, 8, 33, 56, 21, 19, 42, 28, 12	8	Swap 17 with 8	4, 8, 17, 33, 56, 21, 19, 42, 28, 12
3	17, 33, 56, 21, 19, 42, 28, 12	12	Swap 17 with 12	4, 8, 12, 33, 56, 21, 19, 42, 28, 17
4	33, 56, 21, 19, 42, 28, 17	17	Swap 33 with 17	4, 8, 12, 17, 56, 21, 19, 42, 28, 33
5	56, 21, 19, 42, 28, 33	19	Swap 56 with 19	4, 8, 12, 17, 19, 21, 56, 42, 28, 33
6	21, 56, 42, 28, 33	21	No swap	4, 8, 12, 17, 19, 21, 56, 42, 28, 33
7	56, 42, 28, 33	28	Swap 56 with 28	4, 8, 12, 17, 19, 21, 28, 42, 56, 33
8	42, 56, 33	33	Swap 42 with 33	4, 8, 12, 17, 19, 21, 28, 33, 56, 42
9	56, 42	42	Swap 56 with 42	4, 8, 12, 17, 19, 21, 28, 33, 42, 56

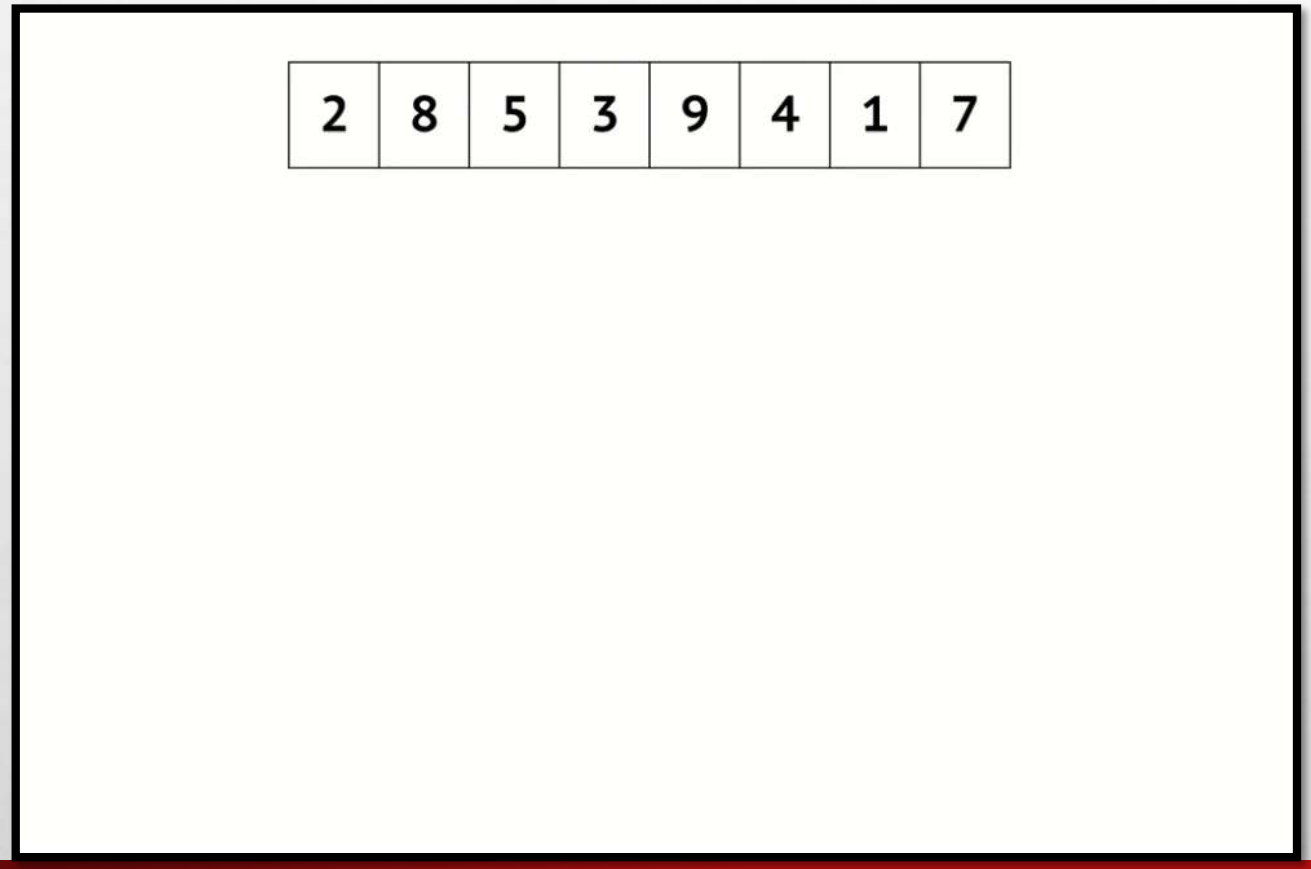
MERGE SORTING

- IT IS A SORTING ALGORITHM THAT IS BASED ON THE DIVIDE AND CONQUER PARADIGM.

"Divide the list into halves, sort each half, then merge them together."



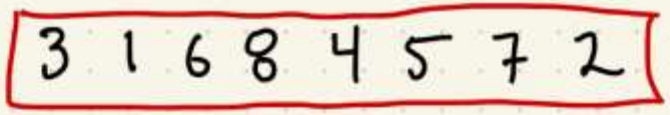
HOW MERGE SORT WORKS?



1. **Divide:**
   Split the array into two halves until each sub-array contains only one element.
2. **Conquer (Sort):**
   Recursively sort each half.
3. **Merge:**
   Combine the sorted halves into one sorted array.

EXAMPLE:

MergeSort:





Will Rosenbaum

EXAMPLE:

Merge Step	Left Array	Right Array	Smaller Element	Result (So Far)
1	10	14	10	[10]
2	–	14	14	[10, 14]
3	29	10, 14	10	[10]
4	29	14	14	[10, 14]
5	29	–	29	[10, 14, 29]
6	37	13	13	[13]
7	37	–	37	[13, 37]
8	10, 14, 29	13, 37	10	[10]
9	14, 29	13, 37	13	[10, 13]
10	14, 29	37	14	[10, 13, 14]
11	29	37	29	[10, 13, 14, 29]
12	–	37	37	[10, 13, 14, 29, 37]

```
//29, 10, 14, 37, 13
```

EXAMPLE:

Merge Step	Left Array	Right Array	Smaller Element	Result (So Far)
1	20	17	17	[17]
2	–	20	20	[17, 20]
3	42	17, 20	17	[17]
4	42	20	20	[17, 20]
5	42	–	42	**[17, 20, 42]**
6	23	50	23	[23]
7	–	50	50	[23, 50]
8	8	23, 50	8	[8]
9	–	23, 50	23	[8, 23]
10	–	50	50	**[8, 23, 50]**
11	17, 20, 42	8, 23, 50	8	[8]
12	17, 20, 42	23, 50	17	[8, 17]
13	20, 42	23, 50	20	[8, 17, 20]
14	42	23, 50	23	[8, 17, 20, 23]
15	42	50	42	[8, 17, 20, 23, 42]
16	–	50	50	**[8, 17, 20, 23, 42, 50]**

```
42, 20, 17, 8, 23, 50
```

TIME COMPLEXITY

Type	Complexity	
Best /Average / Worst Case	O(n log n)	
Space Complexity	O(n)	because it needs extra space for merging

GENERAL ALGORITHM

1. **Compare** the first elements of both arrays.
2. **Copy** the smaller one into the new array.
3. **Repeat** until all elements are merged in order.

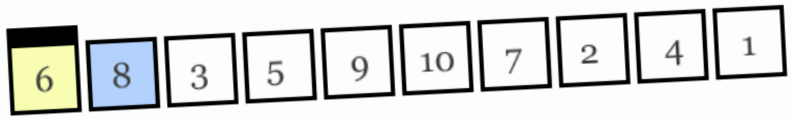
```
MergeSort(array):
   if array has 1 element:
      return array
   else:
      divide array into two halves: left and
right
      left_sorted = MergeSort(left)
      right_sorted = MergeSort(right)
      return Merge(left_sorted,
right_sorted)
```

SELECTION AND MERGE SORTING : LABORATORY

DSAA 211: WEEK 8



SELECTION SORTING : LABORATORY



Yellow is smallest number found
Blue is current item
Green is sorted list

DSAA 211: WEEK 9

PRACTICE 1 : SELECTION SORT

```
    //initialization
    int arr[5] = {29, 10, 14, 37, 13};
    int n = 5;
    int i, j, minIndex, temp;

    //display array
    printf("Original array: ");
    for (i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
```

```c
for (i = 0; i < n - 1; i++) { //outer loop
        minIndex = i;
        for (j = i + 1; j < n; j++) { //inner loop
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }
 //swap smallest element with the first unsorted element
        temp = arr[i];
        arr[i] = arr[minIndex];
        arr[minIndex] = temp;


        printf("After pass %d: ", i + 1);
        for (j = 0; j < n; j++) {
            printf("%d ", arr[j]);
        }
        printf("\n");
    }
```

```c
printf("\nSorted array: ");
    for (i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
```

```
Original array: 29 10 14 37 13

--- Selection Sort Process ---
After pass 1: 10 29 14 37 13
After pass 2: 10 13 14 37 29
After pass 3: 10 13 14 37 29
After pass 4: 10 13 14 29 37

Sorted array: 10 13 14 29 37
```

OUTPUT

Pass	i	Minimum Found	Swap	Array After Pass
1	0	10	29 ↔ 10	10 29 14 37 13
2	1	13	29 ↔ 13	10 13 14 37 29
3	2	14	(no swap)	10 13 14 37 29
4	3	29	37 ↔ 29	10 13 14 29 37

```
Original array: 29 10 14 37 13

--- Selection Sort Process ---
After pass 1: 10 29 14 37 13
After pass 2: 10 13 14 37 29
After pass 3: 10 13 14 37 29
After pass 4: 10 13 14 29 37

Sorted array: 10 13 14 29 37
```

PRACTICE 2 : SELECTION SORT WITH ASCENDING/DESCENDING OPTION

```c
int arr[5];
int n = 5;
int i, j, index, temp, swaps = 0;
char order;

printf("=== Selection Sort ===\n\n");
printf("Enter 5 numbers:\n");
for (i = 0; i < n; i++) {
    printf("Number %d: ", i + 1);
    scanf("%d", &arr[i]);
}
```

```c
//ask user for sorting order
    printf("\nSort in ascending or descending order? (A/D): ");
    scanf(" %c", &order);

    printf("\nOriginal array: ");
    for (i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
```

```
=== Selection Sort ===

Enter 5 numbers:
Number 1: 6
Number 2: 0
Number 3: 3
Number 4: 1
Number 5: 5

Sort in ascending or descending order? (A/D): A

Original array: 6 0 3 1 5
```

```c
printf("\n\n--- Sorting Process ---\n");

//Selection Sort
for (i = 0; i < n - 1; i++) {
    index = i;

    //Find min or max depending on user choice
    for (j = i + 1; j < n; j++) {
        if ((order == 'A' || order == 'a') && arr[j] < arr[index]) {
            index = j;
        } else if ((order == 'D' || order == 'd') && arr[j] > arr[index]) {
            index = j;
        }
    }

    //Swap only if needed
    if (index != i) {
        temp = arr[i];
        arr[i] = arr[index];
        arr[index] = temp;
        swaps++;
    }
```

```
        printf("After pass %d (selected %d): ", i +
1, arr[i]);
        for (j = 0; j < n; j++) {
            printf("%d ", arr[j]);
        }
        printf("\n");
    }

    //display final sorted array
    printf("\nSorted array (%s): ", (order == 'A'
|| order == 'a') ? "Ascending" : "Descending");
    for (i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\nTotal swaps made: %d\n", swaps);
```

```
=== Selection Sort ===

Enter 5 numbers:
Number 1: 0
```

EXERCISE 1: SELECTION SORT

Sort the following value using Selection Sort in ascending order:

45, 12, 88, 32, 5, 60, 19, 25

```
=== SELECTION SORT PROGRAM ===

Original array: 45 12 88 32 5 60 19 25

--- Sorting Process ---
After pass 1: 5 12 88 32 45 60 19 25
After pass 2: 5 12 88 32 45 60 19 25
After pass 3: 5 12 19 32 45 60 88 25
After pass 4: 5 12 19 25 45 60 88 32
After pass 5: 5 12 19 25 32 60 88 45
After pass 6: 5 12 19 25 32 45 88 60
After pass 7: 5 12 19 25 32 45 60 88

Sorted array (Ascending): 5 12 19 25 32 45 60 88
```

EXERCISE 2: SELECTION SORT

Sort the following value using Selection
Sort in descending order:

42, 17, 8, 33, 56, 21, 19, 4, 28, 12

```
=== SELECTION SORT PROGRAM ===

Original array: 42 17 8 33 56 21 19 4 28 12

--- Sorting Process ---
After pass 1: 56 17 8 33 42 21 19 4 28 12
After pass 2: 56 42 8 33 17 21 19 4 28 12
After pass 3: 56 42 33 8 17 21 19 4 28 12
After pass 4: 56 42 33 28 17 21 19 4 8 12
After pass 5: 56 42 33 28 21 17 19 4 8 12
After pass 6: 56 42 33 28 21 19 17 4 8 12
After pass 7: 56 42 33 28 21 19 17 4 8 12
After pass 8: 56 42 33 28 21 19 17 12 8 4
After pass 9: 56 42 33 28 21 19 17 12 8 4

Sorted array (Descending): 56 42 33 28 21 19 17 12 8 4
```

MERGE SORTING : LABORATORY



DSAA 211: WEEK 9

PRACTICE 1 : MERGE SORT

```
    //initialization
    int arr[5] = {29, 10, 14, 37, 13};
    int n = 5;
    int size, leftStart, mid, rightEnd;
    int temp[5];
    int i, j, k;
    //display
    printf("Original array: ");
    for (i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
```

```c
// size = current size of subarrays to merge
    for (size = 1; size < n; size = size * 2) {
        for (leftStart = 0; leftStart < n - 1; leftStart += 2 * size) {
            mid = leftStart + size - 1;
            rightEnd = leftStart + 2 * size - 1;
            // make sure rightEnd does not go beyond the array
            if (rightEnd >= n) {
                rightEnd = n - 1;
            }

            i = leftStart; //LEFT
            j = mid + 1; //RIGHT
            k = leftStart; //START NG MERGE

            // merge the two halves
            while (i <= mid && j <= rightEnd) {
                if (arr[i] < arr[j]) {
                    temp[k] = arr[i];
                    i++;
                } else {
                    temp[k] = arr[j];
                    j++;
                }
                k++;
            }
            // copy remaining elements from left half
            while (i <= mid) {
                temp[k] = arr[i];
                i++;
                k++;
            }
            // copy remaining elements from right half
            while (j <= rightEnd) {
                temp[k] = arr[j];
                j++;
                k++;
            }
            // copy merged result back to original array
            for (i = leftStart; i <= rightEnd; i++) {
                arr[i] = temp[i];
            }
```

```c
printf("\nSorted array: ");
    for (i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
```

```
Original array: 29 10 14 37 13

=== Merge Sort Process ===
After merging: 10 29 14 37 13
After merging: 10 29 14 37 13
After merging: 10 14 29 37 13
After merging: 10 13 14 29 37

Sorted array: 10 13 14 29 37
```

OUTPUT

EXERCISE 1: MERGE SHORT

Sort the following value using Merge Sort, make sure to display the left and right part

45 12 88 32 5 60 19 25

```
Original array: 45 12 88 32 5 60 19 25

--- Merge Sort Process ---

Left part: 45
Right part: 12
Merged result: 12 45

Left part: 88
Right part: 32
Merged result: 32 88

Left part: 5
Right part: 60
Merged result: 5 60

Left part: 19
Right part: 25
Merged result: 19 25

Left part: 12 45
Right part: 32 88
Merged result: 12 32 45 88

Left part: 5 60
Right part: 19 25
Merged result: 5 19 25 60

Left part: 12 32 45 88
Right part: 5 19 25 60
Merged result: 5 12 19 25 32 45 60 88

Final sorted array: 5 12 19 25 32 45 60 88
```

THANK YOU!